:integer	int	int(11)	integer	number(38)
:string	varchar(255)	varchar(255)	char(4096)	varchar2(255)
:text	clob(32768)	text	text	clob
:time	time	time	time	date
:timestamp	timestamp	datetime	timestamp	date

	postgresql	sqlite	sqlserver	sybase
:binary	bytea	blob	image	image
:boolean	boolean	boolean	bit	bit
:date	date	date	date	datetime
:datetime	timestamp	datetime	datetime	datetime
:decimal	decimal	decimal	decimal	decimal
:float	float	float	float(8)	float(8)
:integer	integer	integer	int	int
:string	(note 1)	varchar(255)	varchar(255)	varchar(255)
:text	text	text	text	text
:time	time	datetime	time	time
:timestamp	timestamp	datetime	datetime	timestamp

Using these tables, you could work out that a column declared to be `:integer` in a migration would have the underlying type `integer` in SQLite 3 and `number(38)` in Oracle.

You can use three options when defining most columns in a migration; decimal columns take an additional two options. Each of these options is given as a `key: value` pair. The common options are as follows:

`null: true` or `false`
: If `false`, the underlying column has a `not null` constraint added (if the database supports it). Note that this is independent of any `presence: true` validation, which may be performed at the model layer.

`limit: size`
: This sets a limit on the size of the field. It appends the string `(size)` to the